basic-master-module 코드 리뷰

1. 프로젝트 개요

- 한 줄 설명: `create-next-app` 으로 막 생성한 Next.js 16 App Router 기본 골격. 이름("master module")으로 보아 워크스페이스 내 Next.js + Supabase 계열 프로젝트들의 공통 출발점으로 쓰려던 것으로 보인다.
- 목적/대상 사용자: 대상은 개발자 본인과 AI 코딩 에이전트다. 새 프로젝트를 시작할 때 복제해 쓰는 베이스 템플릿 역할을 노렸다.

영역	내용
프론트	Next.js 16.2.4 (App Router), React 19.2.4, Tailwind CSS v4 (`@tailwindcss/postcss`)
백엔드	없음 (Route Handler, Server Action 없음)
DB	없음 (Supabase 의존성 없음)
인프라	없음 (`.vercel` 이 `gitignore` 에만 있음). git remote DevOps0906/basic_master_module 은 설정됨
AI	없음. `AGENTS.md`/`CLAUDE.md` 는 Next.js 기본 에이전트 규칙만 담고 있음

- 현재 완성도: 공통 템플릿 기준 약 5%. 생성기 기본 파일에서 수정한 것이 하나도 없다. `app/page.tsx` 는 Vercel 홍보 페이지 그대로이고, `app/layout.tsx:16` 의 title 도 "Create Next App" 이다. git 에는 커밋이 한 번도 없다(`README.md` 만 staged, 나머지는 untracked).
- 핵심 기능 목록: 없음. 동작하는 것은 TypeScript strict, ESLint flat config(core-web-vitals + typescript), Tailwind v4, Geist 폰트 로딩, `@/*` 경로 별칭까지다.

2. 구조 분석

경로	역할
`app/layout.tsx`	RootLayout. Geist 폰트 변수와 `lang="en"`, 기본 metadata
`app/page.tsx`	create-next-app 기본 랜딩 페이지 (외부 Vercel 링크)
`app/globals.css`	Tailwind import, `:root` 색상 토큰 2개, 다크모드 미디어쿼리
`app/favicon.ico`	기본 파비콘
`public/*.svg`	next/vercel/file/globe/window 기본 아이콘 5개
`next.config.ts`	빈 설정
`tsconfig.json`	strict, bundler 해석, `@/*` 는 루트(`./*`) 기준
`eslint.config.mjs`	eslint-config-next core-web-vitals + typescript
`AGENTS.md`, `CLAUDE.md`	Next.js 16 버전 경고 1개 (`CLAUDE.md` 는 `@AGENTS.md` 만 import)

데이터 흐름은 없다. 정적 페이지 하나가 RootLayout 안에서 렌더된다.

공통 베이스로서의 위치(형제 프로젝트 package.json 과 디렉토리만 가볍게 비교함):

프로젝트	next	@supabase/ssr	인증 경계 파일	supabase 클라이언트 위치	기타
basic-master-module	16.2.4	없음	없음	없음	-
Hospital	16.2.9	^0.12.0	src/proxy.ts	src/lib/supabase/	shadcn, zustand
School_Management_System	^16.2.9	^0.12.0	middleware.ts	src/lib/supabase/	TypeScript ^6

프로젝트	next	@supabase/ssr	인증 경계 파일	supabase 클라이언트 위치	기타
ERP-Wesite	^16.2.9	^0.6.1	middleware.ts	lib/supabase/	react ^19.0.0
Nomad-BlogStudio	16.2.9	^0.12.0	src/middleware.ts	src/lib/supabase/	zod, react-query, RHF, shadcn
sellingstudio	16.2.7	^0.10.3	proxy.ts	lib/supabase/ (+admin)	zod, react-query, RHF, shadcn, radix-ui

결론: 이 모듈은 지금 어떤 프로젝트의 베이스로도 쓰이지 않고 있다. 다섯 프로젝트가 같은 스택(Next 16 + React 19 + Tailwind 4 + Supabase SSR + clsx/tailwind-merge/lucide)을 쓰지만 각자 따로 구성했다. 그 결과 다음이 제각각이다.

- `@supabase/ssr` 버전: 0.6 ~ 0.12
- 인증 경계 파일: Next 16 의 `proxy.ts` 를 쓴 곳과 deprecated 된 `middleware.ts` 를 쓴 곳이 섞여 있음
- 디렉토리 규칙: `src/` 사용 여부가 다름

반복되는 보일러플레이트가 많으므로 공통 템플릿으로 키울 가치가 충분하다.

3. 발견된 문제점

#	심각도	분류	위치(경로:줄)	내용
1	높음	구조	package.json:11-15	공통 템플릿이라면서 Supabase(@supabase/ssr, supabase-js), 인증 경계(proxy.ts), lib/ 가 전혀 없다. 형제 프로젝트 5개가 매번 직접 구성한다
2	높음	설정	(저장소 전체)	git 커밋이 0개다. README.md 만 staged 이고 app/, 설정 파일은 untracked 다. 템플릿으로 복제하거나 버전을 관리할 기준점이 없다
3	중간	설정	package.json:12,22	next/eslint-config-next 가 16.2.4 로 고정되어 형제 프로젝트(16.2.7~16.2.9)보다 낮다. 템플릿이 가장 오래된 버전이 된다
4	중간	버그	app/globals.css:25	body 에 font-family: Arial, Helvetica, sans-serif 가 직접 지정되어 있다. 그래서 layout.tsx:5-13 에서 로드한 Geist 와 --font-sans(globals.css:11) 가 body 기본 글꼴에 적용되지 않는다. 폰트를 다운로드하고도 쓰지 않는 셈이다
5	중간	설정	.gitignore:34	.env* 패턴이 .env.example 까지 무시한다. 템플릿이 필수 환경변수 목록을 배포할 수 없다 (!.env.example 필요)
6	중간	품질	package.json:5-10	typecheck, test, format 스크립트가 없다. 테스트 러너(Vitest/Playwright)와 Prettier 도 없다. lint 는 eslint 만 실행해 대상 경로가 암묵적이다
7	중간	품질	AGENTS.md:1-5	에이전트 규칙이 "Next 16 문서를 읽어라" 하나뿐이다. proxy.ts 사용, Supabase 클라이언트 생성 규칙, 디렉토리 규칙, 금지사항(service role 키의 클라이언트 노출 등) 같은 워크스페이스 공통 규약이 없다
8	낮음	품질	app/layout.tsx:16-17,27	metadata 가 "Create Next App" 그대로이고 lang="en" 이다. 한국어 서비스 중심 워크스페이스에 맞지 않는다
9	낮음	품질	app/page.tsx:1-65	Vercel 홍보 랜딩(utm 링크 포함)이 그대로 있다. public/*.svg 5개도 쓰임새 없는 기본 에셋이다
10	낮음	설정	next.config.ts:3-5	빈 설정이다. Supabase Storage 이미지용 images.remotePatterns, 보안 헤더(headers()), typedRoutes 등 공통 기본값이 없다
11	낮음	구조	tsconfig.json:21-23	@/* 가 루트 기준이다. 형제 프로젝트 중 3개는 src/ 를 쓴다. 템플릿에서 src/ 사용 여부를 확정하지 않으면 계속 갈라진다

높음 항목 상세

1. Supabase/인증 부재 (#1): 이 워크스페이스에서 "기본 모듈"이 존재해야 하는 이유는 Supabase SSR 클라이언트, 세션 갱신 proxy, 보호 라우트 같은 보안에 민감한 보일러플레이트를 한 번만 올바르게 작성하는 데 있다. 지금은 프로젝트마다 이 코드를 따로 작성한다. 그 결과 `@supabase/ssr` 가 0.6~0.12 로 흩어졌고, Next 16 에서 deprecated 된 `middleware.ts` 가 School_Management_System, ERP-Wesite, Nomad-BlogStudio 에 남아 있다. 템플릿이 이 코드를 제공하지 않으면 버그 수정도 다섯 군데에서 따로 해야 한다.
2. 커밋 없음 (#2): `git status` 를 보면 브랜치 `master` 에 커밋이 없다. remote(DevOps0906/basic_master_module)는 설정되어 있어도 push 된 내용이 없다. 템플릿은 `degit` 이나 GitHub Template Repository 로 복제되어야 하므로, 첫 커밋과 태그(v0.1.0)가 모든 후속 작업의 전제 조건이다.

4. 효율성 개선 제안

- 구조/아키텍처
  - 형제 프로젝트 중 가장 최신 패턴을 쓰는 Hospital(src/proxy.ts, `@supabase/ssr` ^0.12)과 sellingstudio(lib/supabase/admin.ts 분리)의 구성을 기준안으로 추출해 이 모듈로 옮긴다.
  - 표준 구조는 src/app, src/lib/supabase/{client,server,admin}.ts, src/proxy.ts, src/components/ui, src/types/supabase.ts 로 고정한다.
  - 라우트 그룹 (auth)/(app) 과 로그인, 콜백(auth/callback/route.ts), 로그아웃 Server Action 까지 포함한다.
- 성능
  - 템플릿 단계에서는 거의 해당 사항이 없다.
  - Geist 폰트가 실제로 적용되도록 globals.css:25 를 수정하고, 쓰지 않는 `Geist_Mono` 는 필요할 때만 로드한다.
  - `next/image` 원격 패턴은 Supabase Storage 도메인으로 한정한다.
- 개발 생산성
  - 공통 의존성 묶음(zod, react-hook-form, @tanstack/react-query, zustand, clsx, tailwind-merge, lucide-react, shadcn)의 버전을 형제 프로젝트 최신값으로 맞춘다.
  - `typecheck`(tsc --noEmit), `test`(Vitest), `e2e`(Playwright), `format`(Prettier) 스크립트를 추가한다.
  - `supabase gen types` 스크립트로 타입 생성을 표준화한다.
- 운영/배포
  - .env.example(URL, publishable key, service role key 는 서버 전용으로 명시)과 `zod` 기반 env 검증 모듈을 둔다.
  - GitHub Actions(lint + typecheck + test + build)를 둔다.
  - GitHub Template Repository 로 지정하고 버전 태그로 관리한다.

권고: **템플릿으로 통합**. 이 모듈 자체는 내용이 없어 "계속 발전"시킬 대상이 아니다. 형제 프로젝트에서 검증된 Supabase/인증/UI 골격을 이 저장소로 모아 공식 공통 템플릿으로 승격하는 것이 맞다. 아카이브하면 프로젝트별 편차가 계속 커진다.

5. 수정 작업 목록 및 권장 모델

#	작업	우선순위	예상 규모	권장 모델	선정 이유
1	기본 브랜치를 `main` 으로 바꾸고 현재 파일을 첫 커밋으로 올린 뒤 push 하고 v0.1.0 태그를 단다	P0	S	Sonnet 5.5	단순 git 작업
2	next/eslint-config-next 를 16.2.9 로 올리고 @types/node 를 프로젝트 Node 버전에 맞춘다. engines 필드를 추가한다	P0	S	Sonnet 5.5	설정 파일 정리

#	작업	우선순위	예상 규모	권장 모델	선정 이유
3	src/ 구조로 이전하고 tsconfig paths 를 ./src/* 로 바꾼다. 기본 랜딩, public/*.svg 를 제거하고 metadata/lang="ko" 를 설정한다. globals.css:25 폰트 버그를 수정한다	P0	S	Sonnet 5.5	보일러플레이트 정리
4	@supabase/ssr ^0.12 와 supabase-js 를 추가하고 src/lib/supabase/client.ts, server.ts(cookies 비동기 API), admin.ts(server-only, service role) 를 작성한다. Hospital/sellingstudio 구현을 기준으로 한다	P0	M	Fable 5.1	인증/키 노출 경계로 실수 비용이 큼
5	src/proxy.ts 를 작성한다. 세션 갱신, 보호 경로 리다이렉트, matcher 에서 정적 자원 제외를 포함하고, getUser() 기반 검증과 쿠키 전달 누락 방지를 반영한다	P0	M	Fable 5.1	인증 로직, Next 16 proxy 규약
6	인증 흐름을 만든다. (auth)/login 페이지, auth/callback/route.ts, 로그아웃 Server Action, (app) 보호 레이아웃(서버에서 사용자 확인)	P1	M	Opus 5.5	정의된 중간 규모 기능, 5와 연동
7	.gitignore 에 !.env.example 을 추가하고 .env.example 과 src/lib/env.ts(zod 로 서버/클라이언트 env 분리 검증)를 작성한다	P1	S	Sonnet 5.5	설정 및 보일러플레이트
8	shadcn 을 초기화(components.json)하고 공통 UI(Button, Input, Card, Dialog, Toast)와 cn() 유틸, 기본 AppShell(사이드바 + 헤더) 레이아웃을 추가한다	P1	M	Sonnet 5.5	반복적 컴포넌트 작업
9	공통 의존성(zod, react-hook-form, @tanstack/react-query, zustand, lucide-react)을 추가하고 providers.tsx(QueryClientProvider) 를 RootLayout 에 연결한다	P1	S	Opus 5.5	상태관리 기본 구조 결정
10	Vitest + Testing Library, Playwright 를 설정하고 proxy 리다이렉트와 로그인 페이지 스모크 테스트를 작성한다	P1	M	Opus 5.5	테스트 설계
11	package.json 에 typecheck/test/e2e/format/db:types(supabase gen types) 스크립트와 Prettier(tailwind 플러그인) 설정을 추가한다	P2	S	Sonnet 5.5	설정 정리
12	next.config.ts 에 Supabase Storage remotePatterns 와 보안 헤더(CSP 초안, X-Frame-Options, Referrer-Policy)를 넣는다	P2	S	Opus 5.5	보안 헤더는 오설정 시 기능 장애
13	GitHub Actions CI(install, lint, typecheck, test, build)를 추가하고 저장소를 Template Repository 로 지정한다	P2	S	Sonnet 5.5	보일러플레이트
14	AGENTS.md 에 워크스페이스 공통 규약을 정리한다(디렉토리 구조, Supabase 클라이언트 선택 규칙, proxy 사용, service role 금지 영역, 커밋 규칙). README 에 템플릿 사용법을 쓴다	P2	S	Sonnet 5.5	문서화

6. 종합 평가

- 강점
  1. 최신 스택(Next 16, React 19, Tailwind 4, ESLint 9 flat config)과 TypeScript strict 가 이미 잡혀 있다.
  2. 아무것도 없는 깨끗한 상태라 기존 프로젝트들의 모범 패턴만 골라 담을 수 있다.
  3. AGENTS.md/CLAUDE.md 연결 구조가 있어 에이전트 규약을 넣을 자리가 마련되어 있다.
- 약점
  1. 공통 템플릿에 필요한 Supabase/인증/레이아웃/UI 가 전무하다.
  2. 커밋이 없어 버전 관리와 복제 기준점이 없다.
  3. 테스트, 타입체크, CI, env 관리가 없고, 버전이 형제 프로젝트보다 오래되었다.
- 모델별 비율 (14개 작업 기준): Fable 5.1 약 14% (2개) / Opus 5.5 약 29% (4개) / Sonnet 5.5 약 57% (8개)
- 권장 진행 순서
  - 1단계: 기반 정리. 작업 1, 2, 3, 7
  - 2단계: Supabase 와 인증 골격. 작업 4, 5, 6 (Fable 이 4·5 를 끝낸 뒤 Opus 가 6 을 진행)

    - 3단계: 생산성과 배포. 작업 8~14. 이후 형제 프로젝트의 `middleware.ts` 를 `proxy.ts` 로 옮길 때 이 템플릿을 기준으로 삼는다
- 주 담당 모델: **Opus 5.5**. 작업 수로는 Sonnet 비중이 가장 크지만, 대부분 설정이나 정리성 작업이라 짧게 끝난다. 이 프로젝트의 가치는 형제 프로젝트 5개의 구현을 비교해 하나의 기준안으로 추출하고 일관되게 묶는 데 있다. 인증 흐름 연결, 상태관리 구조, 테스트 설계처럼 2~5개 파일에 걸친 중간 규모 판단이 중심이므로 Opus 가 주 담당으로 가장 적합하다. 다만 Supabase 클라이언트 경계(작업 4)와 proxy 인증 로직(작업 5)은 템플릿을 통해 모든 프로젝트로 퍼지는 보안 코드이므로 Fable 에 맡긴다.